Cryptology

LAB Assignment 7

Name: Shreyas S Sanil

Roll No: 261100690003

Subject: Cryptology

Columnar Transposition, Double Columnar Transposition and OpenSSL

Question 1(a): Columnar Transposition Cipher — Encryption and Decryption

The plaintext is written in rows under a keyword. The columns are then read out in the alphabetical order of the keyword's letters to form the ciphertext. Padding ('X') is added so the grid is rectangular. Decryption rebuilds the grid column by column in that same key order, then reads it row by row.

```
def get_key_order(key):
    return sorted(range(len(key)), key=lambda i: key[i])
def columnar_encrypt(text, key):
    text = text.replace(" ", "").upper()
    cols = len(key)
    rows = -(-len(text) // cols)
    padded = text.ljust(rows * cols, "X")
    grid = [padded[r*cols:(r+1)*cols] for r in range(rows)]
    order = get_key_order(key)
    cipher = ""
    for col in order:
        cipher += "".join(row[col] for row in grid)
    return cipher
def columnar_decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher) // cols
    order = get_key_order(key)
    grid = [["" for _ in range(cols)] for _ in range(rows)]
    idx = 0
    for col in order:
        for r in range(rows):
            grid[r][col] = cipher[idx]
            idx += 1
    return "".join("".join(row) for row in grid)
plaintext = input("Enter plaintext: ")
key = input("Enter keyword: ").upper()
cipher = columnar_encrypt(plaintext, key)
print("Columnar Encrypted:", cipher)
print("Columnar Decrypted:", columnar_decrypt(cipher, key))
```

Sample Run (plaintext = "ATTACKATDAWN", key = "ZEBRA")

```
Enter plaintext: ATTACKATDAWN
Enter keyword: ZEBRA
Columnar Encrypted: CAXTTXTANADXAKW
Columnar Decrypted: ATTACKATDAWNXXX
```

Question 1(b): Double Columnar Transposition Cipher — Encryption and Decryption

Double columnar transposition applies the columnar transposition twice, with two different keywords, so the single-pass pattern above is itself scrambled a second time, which makes the letter positions harder to recover.

```
def double_columnar_encrypt(text, key1, key2):
    first_pass = columnar_encrypt(text, key1)
    return columnar_encrypt(first_pass, key2)
def double_columnar_decrypt(cipher, key1, key2):
    first_pass = columnar_decrypt(cipher, key2)
    return columnar_decrypt(first_pass, key1)
plaintext = input("Enter plaintext: ")
key1 = input("Enter first keyword: ").upper()
key2 = input("Enter second keyword: ").upper()
cipher = double_columnar_encrypt(plaintext, key1, key2)
print("Double Columnar Encrypted:", cipher)
print("Double Columnar Decrypted:", double_columnar_decrypt(cipher, key1, key2))
```

Sample Run (plaintext = "ATTACKATDAWN", key1 = "ZEBRA", key2 = "KEY")

```
Enter plaintext: ATTACKATDAWN
Enter first keyword: ZEBRA
Enter second keyword: KEY
Double Columnar Encrypted: ATADKCTTAAXXNXW
Double Columnar Decrypted: ATTACKATDAWNXXX
```

The decrypted text carries three trailing 'X' padding characters, which is expected: the first pass needed 15 grid cells for a 12-letter word under a 5-letter key, and this padding is recovered along with the message.

Bonus: Brute-Force Cryptanalysis of Columnar Transposition

Because a columnar transposition with a short keyword has few possible column orders, all of them can simply be tried. Each candidate decryption is scored by how many common English bigrams ("TH", "HE", "IN", ...) it contains, and the highest-scoring order is reported as the likely key.

```
import itertools
COMMON_BIGRAMS = ["TH", "HE", "IN", "ER", "AN", "RE", "ON", "AT", "EN",
                  "ND", "TI", "ES", "OR", "TE", "OF", "ED", "IS", "IT"]
```

```
def score_text(text):
    text = text.upper()
    return sum(text.count(bg) for bg in COMMON_BIGRAMS)
def columnar_decrypt_order(cipher, order):
    cols = len(order)
    rows = len(cipher) // cols
    grid = [["" for _ in range(cols)] for _ in range(rows)]
    idx = 0
    for col in order:
        for r in range(rows):
            grid[r][col] = cipher[idx]
            idx += 1
    return "".join("".join(row) for row in grid)
def brute_force_columnar(cipher, num_cols):
best = None
for perm in itertools.permutations(range(num_cols)):
    candidate = columnar_decrypt_order(cipher, list(perm))
    s = score_text(candidate)
    if best is None or s > best[0]:
    best = (s, perm, candidate)
return best
ciphertext = input("Enter ciphertext: ")
num_cols = int(input("Enter number of columns to try: "))
score, order, guess = brute_force_columnar(ciphertext, num_cols)
print(f"Best column order found : {order}")
print(f"Best guess at plaintext : {guess}")
```

Sample Run

```
Enter ciphertext: HETKDXEYAAWXTNACTNEMTSAX
Enter number of columns to try: 4
Best column order found : (1, 2, 0, 3)
Best guess at plaintext : THEENEYMATATCKASTDWANXXX
```

This is close to the real plaintext ("THE ENEMY ATTACKS AT DAWN") but not an exact match, because bigram scoring alone cannot always break a tie on short ciphertext. The attack is still practical for key lengths up to 6–7 columns (where 5040–720 permutations can be checked in seconds) and demonstrates why classical transposition ciphers are considered weak against modern automated cryptanalysis — it is exactly why double columnar transposition (Q1(b)) and longer keywords are used to raise the number of permutations beyond what brute force can search.

Question 2: Exploring the OpenSSL Tool

OpenSSL is a free, open-source command-line toolkit and library for TLS/SSL and general-purpose cryptography. It provides symmetric and asymmetric ciphers, hash functions, key and certificate management, and tools to test TLS connections. The commands below were run in a terminal to explore it for this lab.

2.1 Checking the installed version

```
$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

2.2 Symmetric encryption (AES-256-CBC) of a file

The enc sub-command performs symmetric encryption. -pbkdf2 derives the key from a password using PBKDF2 instead of the older, weaker key derivation.

```
$ echo "Cryptology Lab - OpenSSL demo" > plain.txt
$ openssl enc -aes-256-cbc -pbkdf2 -salt -in plain.txt -out cipher.enc -k mypassword
$ openssl enc -aes-256-cbc -pbkdf2 -d -in cipher.enc -out decrypted.txt -k mypassword
$ diff plain.txt decrypted.txt
# (no output - files are identical, confirming correct decryption)
```

2.3 Computing hashes

```
$ openssl dgst -sha256 plain.txt
SHA2-256(plain.txt)= 2c7a1e... (64 hex characters)
$ openssl dgst -md5 plain.txt
MD5(plain.txt)= 9f1b3c... (32 hex characters)
```

2.4 Generating an RSA key pair and a self-signed certificate

```
$ openssl genrsa -out private.pem 2048
$ openssl rsa -in private.pem -pubout -out public.pem
$ openssl req -new -x509 -key private.pem -out cert.pem -days 365 \
  -subj "/CN=cryptology-lab/O=MSIS"
$ openssl x509 -in cert.pem -text -noout | head -5
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: ...
        Signature Algorithm: sha256WithRSAEncryption
```

2.5 Testing a TLS connection

```
$ openssl s_client -connect example.com:443 -brief
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
```

> Ciphersuite: TLS_AES_256_GCM_SHA384
> Verification: OK

2.6 Summary of findings

• enc performs symmetric encryption/decryption (AES, ChaCha20 and others) directly from the command line. • dgst computes message digests (SHA-256, SHA-1, MD5) used for integrity checks. • genrsa, rsa and req create RSA key pairs and X.509 certificates, which are the basis of TLS/SSL. • s_client opens a live TLS connection to a server and reports the negotiated protocol version and cipher suite, useful for verifying a server's security configuration. • OpenSSL is therefore both a practical encryption/hashing utility and the library mos